Financial Econometrics Practice TS #2 (Forecasting)

Daniel Campos, Joshua Castillo, & Cherryl Chico; February 22, 2026

Contents

1 Load Packages and Data **1**
1.1 Plotting the series . . . 2

2 Transformations **3**
2.1 Functions . . . 3
2.2 Variance and Mean Stability Check . . . 4
2.3 LN and Box-Cox Transformations . . . 5
2.4 ACF and PACF Analysis . . . 6

3 SARIMA Models **8**
3.1 Functions . . . 8
3.2 Models . . . 11
3.3 Run Analysis . . . 12
3.4 Choosing Model . . . 15

4 Model Shortlist **15**
4.1 Function . . . 15
4.2 Final Model: ARIMA(1,2,1)(0,1,1)[12] . . . 23

1 Load Packages and Data

```r
library(forecast)
library(tseries)
library(lmtest)
library(nortest)
library(MASS)
```

We read the unemployment data from the Spanish Employment Public Service (SEPE), representing the count of individuals registered as unemployed at SEPE offices. The dataset spans from January 2000 to present, sourced from https://expinterweb.mites.gob.es/series/. The data is converted into a time series object with a starting window from 2000.

```r
series <- window(
  ts(read.table("unemploy.dat", dec = ",") / 1e6,
     start = 1996, freq = 12),
  start = c(2000),
  end = c(2025, 12)
)

# Display properties
cat("Class:", class(series), "\n")
cat("Length:", length(series), "observations\n")
```

```r
cat("Start:", start(series)[1], "Period", start(series)[2], "\n")
cat("End:", end(series)[1], "Period", end(series)[2], "\n")
cat("Frequency:", frequency(series), "\n\n")

# Display first five years of data
head(round(series, 3),12*5)
```

```
## Class: ts
## Length: 312 observations
## Start: 2000 Period 1
## End: 2025 Period 12
## Frequency: 12
##
##        Jan   Feb   Mar   Apr   May   Jun   Jul   Aug   Sep   Oct   Nov   Dec
## 2000 2.090 2.080 2.042 1.984 1.929 1.903 1.893 1.888 1.908 1.934 1.963 1.947
## 2001 2.017 1.993 1.981 1.910 1.898 1.843 1.836 1.879 1.889 1.941 1.986 1.989
## 2002 2.075 2.150 2.083 2.060 2.003 1.963 1.962 1.984 2.007 2.065 2.117 2.127
## 2003 2.185 2.180 2.163 2.104 2.036 2.020 1.996 2.017 2.040 2.097 2.143 2.181
## 2004 2.233 2.219 2.182 2.162 2.091 2.054 2.014 2.050 2.051 2.076 2.121 2.113
```

```r
# constants
series_name <- "Unemployment Rate"
start_year <- 2000
start_period <- 1
freq <- 12
period_length <- 12
xlabel <- "month"
ylabel <- "Unemployment Rate (%)"
```

1.1 Plotting the series

```r
plot(series, main=series_name, xlab=xlabel, ylab=ylabel)
abline(v=1996:2026,col=4,lty=3)
```

The data exhibits clear yearly seasonal patterns with peaks at end of the year and troughs in the middle of the year, likely due to seasonal employment patterns. We also see a sharp increase during the 2008 financial crisis, followed by a gradual decline until 2020, and then another sharp increase during the COVID-19 pandemic.

To fit a SARIMA model, we transform the series to achieve mean and variance stability. We begin with a preliminary visual evaluation of stationarity by analyzing rolling mean and variance. This approach provides a continuous view of potential changes, avoiding the artifacts introduced by binning.

2 Transformations

2.1 Functions

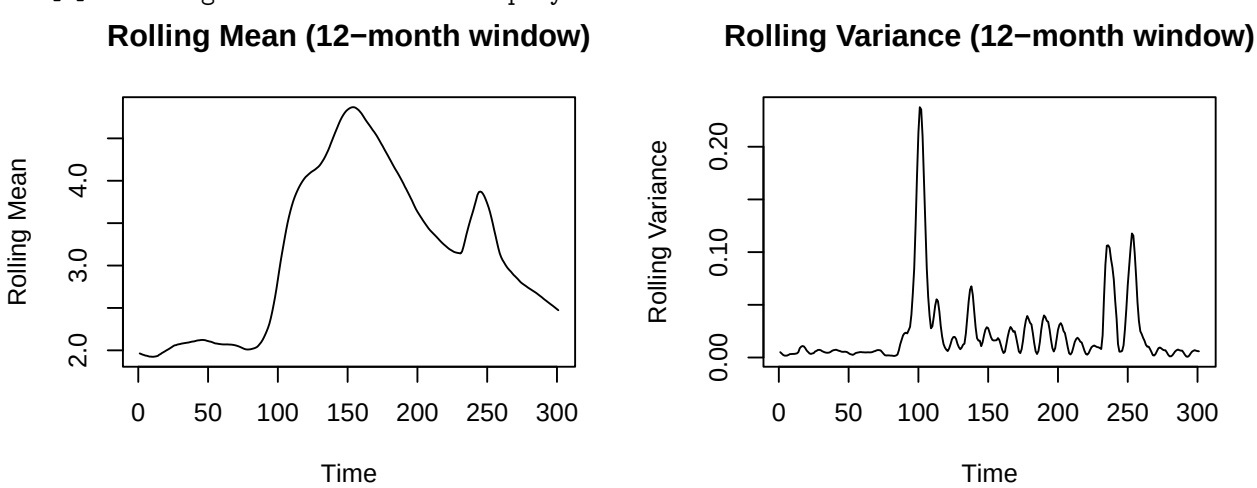
```
# function to get rolling ts
get_rolling_ts <- function(series, window_size) {
  rolling_mean <- sapply(1:(length(series) - window_size + 1), function(i) {
    mean(series[i:(i + window_size - 1)])
  })
  rolling_var <- sapply(1:(length(series) - window_size + 1), function(i) {
    var(series[i:(i + window_size - 1)])
  })

  data.frame(
    rolling_mean = rolling_mean,
    rolling_variance = rolling_var
  )
}

produce_plots <- function(series, name="Series") {
  rolling_stats <- get_rolling_ts(series, freq)
  print(paste("Rolling statistics for:", name))
  par(mfrow=c(2,2))
```

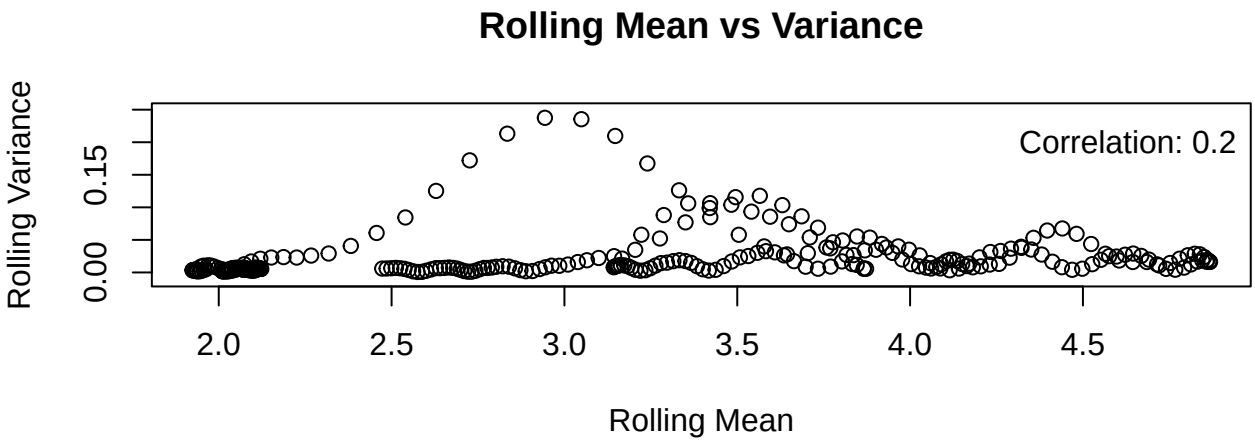
```r
  # Plot 1: Rolling Mean over time
  plot(rolling_stats$rolling_mean, type="l",
       main = paste("Rolling Mean (", freq, "-month window)", sep = ""),
       xlab = "Time", ylab = "Rolling Mean")

  # Plot 2: Rolling Variance over time
  plot(rolling_stats$rolling_variance, type="l",
       main = paste("Rolling Variance (", freq, "-month window)", sep = ""),
       xlab = "Time", ylab = "Rolling Variance")

  # Plot 3 & 4: Mean vs Variance (spanning full width)
  par(mfrow=c(2,1), oma=c(0, 0, 2, 0))
  y_range <- range(rolling_stats$rolling_variance)
  y_padding <- (y_range[2] - y_range[1]) * 0.05
  plot(rolling_stats$rolling_mean, rolling_stats$rolling_variance,
       main = "Rolling Mean vs Variance",
       xlab = "Rolling Mean", ylab = "Rolling Variance",
       ylim = c(y_range[1] - y_padding, y_range[2] + y_padding))

  # print correlation
  cor_val <- cor(rolling_stats$rolling_mean, rolling_stats$rolling_variance)
  legend("topright", legend = paste("Correlation:", round(cor_val, 3)), bty = "n")

  par(mfrow=c(1,1))
}
```

2.2 Variance and Mean Stability Check

```r
produce_plots (series, series_name)
```

```
## [1] "Rolling statistics for: Unemployment Rate"
```





We can see that the rolling mean is not stable, with clear changes in trend over time. The rolling variance also shows significant fluctuations, particularly during the 2008 financial crisis and the COVID-19 pandemic. The correlation between rolling mean and variance is low (0.08), suggesting that the variance does not strongly depend on the mean, but the presence of heteroskedasticity is still evident from the variance plot.

We first address the variance instability with transformations, and then we handle the mean instability with differencing prior to fitting the SARIMA model.

2.3 LN and Box-Cox Transformations

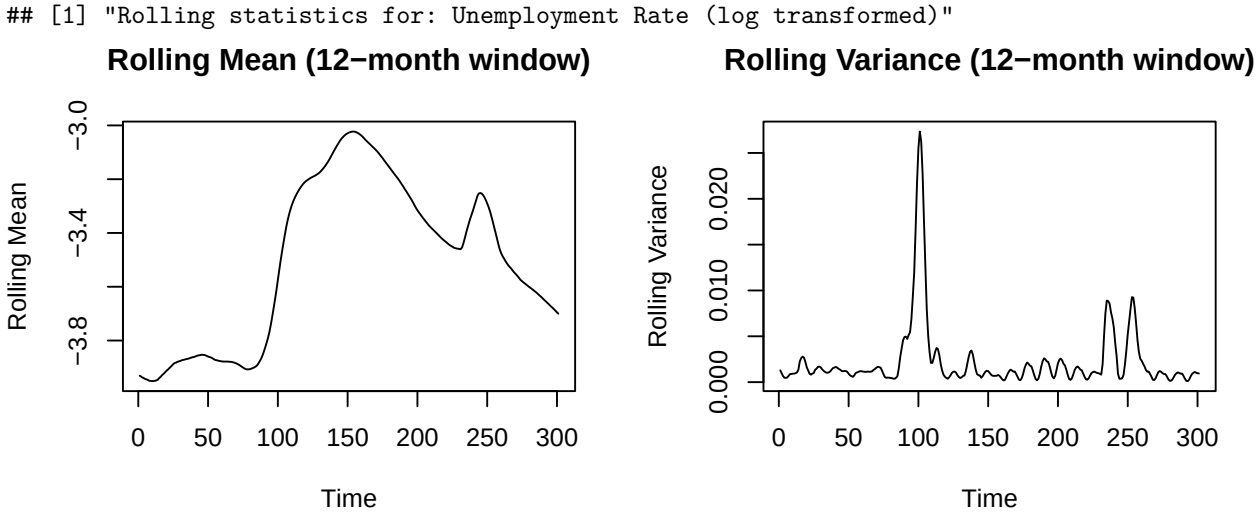
```
ln_transformed <- log(series / 100)

# Box-Cox transformation
boxcox_result <- boxcox(series ~ 1, lambda = seq(-2, 2, by = 0.01), plot=FALSE)
optimal_lambda <- boxcox_result$x[which.max(boxcox_result$y)]
cat("Optimal lambda for Box-Cox transformation:", optimal_lambda, "\n")
# Apply Box-Cox transformation with optimal lambda
bc_transformed <- (series^optimal_lambda - 1) / optimal_lambda
```

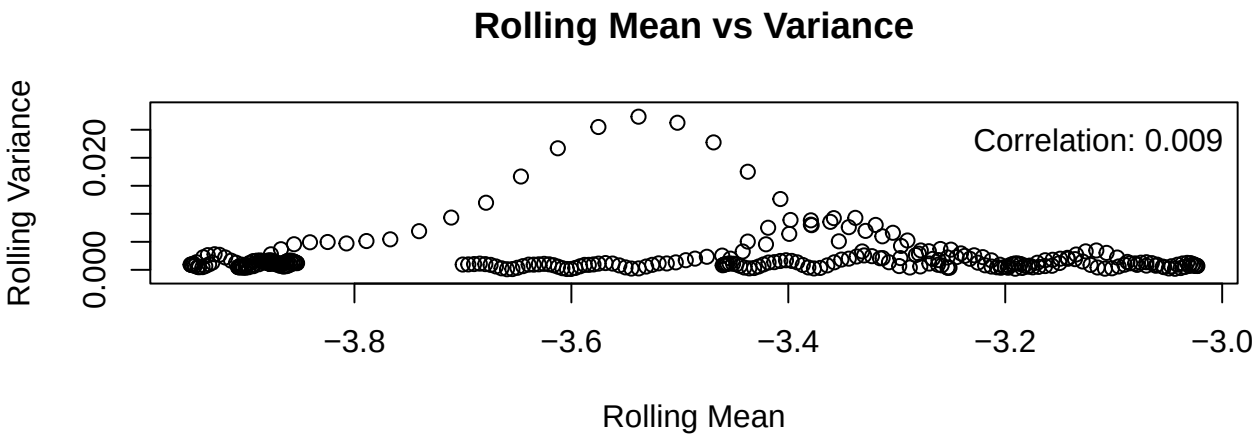
```
## Optimal lambda for Box-Cox transformation: -0.03
```

Because optimal lambda for the Box-Cox transformation is close to 0, the log transformation is likely sufficient for variance stabilization. Thus, we proceed to investigate the plots only for the log transformation.

```
produce_plots (ln_transformed, paste(series_name, "(log transformed)"))
```

```
## [1] "Rolling statistics for: Unemployment Rate (log transformed)"
```





Transforming the data with the natural logarithm reduced the overall level but did not stabilize the variance. The 2008 financial crisis and the COVID-19 pandemic still show significant spikes in variance, which suggest that there could be structural breaks in the data.

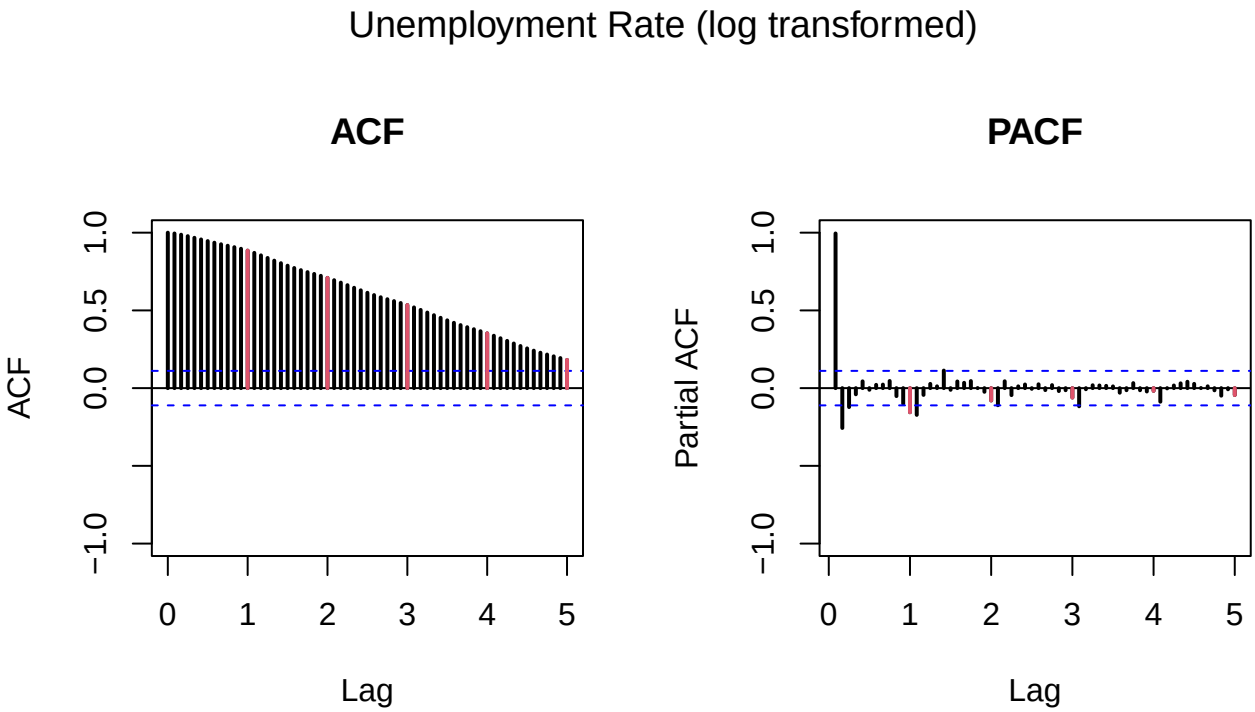
For this exercise, we keep as is as handling structural breaks is beyond the scope of this assignment.

2.4 ACF and PACF Analysis

Using the log-transformed series, we analyze the ACF and PACF plots to determine the appropriate order of differencing and the potential AR and MA terms for the SARIMA model.

```
acf_pacf_plots <- function(series, name="Series") {
  s <- frequency(series)
  lag_limit <- max(60, 3*s)

  acf_obj <- acf(series, plot=FALSE, lag.max=lag_limit)
  pacf_obj <- pacf(series, plot=FALSE, lag.max=lag_limit)

  par(mfrow=c(1,2), oma=c(0, 0, 2, 0))
  plot(acf_obj, main = "ACF", ylim=c(-1,1), lwd=2, col=1)

  acf_lags <- as.numeric(acf_obj$lag)[-1]
  acf_vals <- as.numeric(acf_obj$acf)[-1]
  seasonal_lags <- seq(s, lag_limit, by=s)

  acf_idx <- which(round(acf_lags * s) %in% seasonal_lags)
  segments(acf_lags[acf_idx], 0, acf_lags[acf_idx], acf_vals[acf_idx], col=2, lwd=2)
  plot(pacf_obj, main = "PACF", ylim=c(-1,1), lwd=2, col=1)

  pacf_lags <- as.numeric(pacf_obj$lag)
  pacf_vals <- as.numeric(pacf_obj$acf)
  pacf_idx <- which(round(pacf_lags * s) %in% seasonal_lags)
  segments(pacf_lags[pacf_idx], 0, pacf_lags[pacf_idx], pacf_vals[pacf_idx], col=2, lwd=2)

  mtext(name, outer=TRUE, cex=1.2, line=0.5)
  par(mfrow = c(1, 1))
}
```

```r
acf_pacf_plots(ln_transformed, paste(series_name, "(log transformed)"))
```



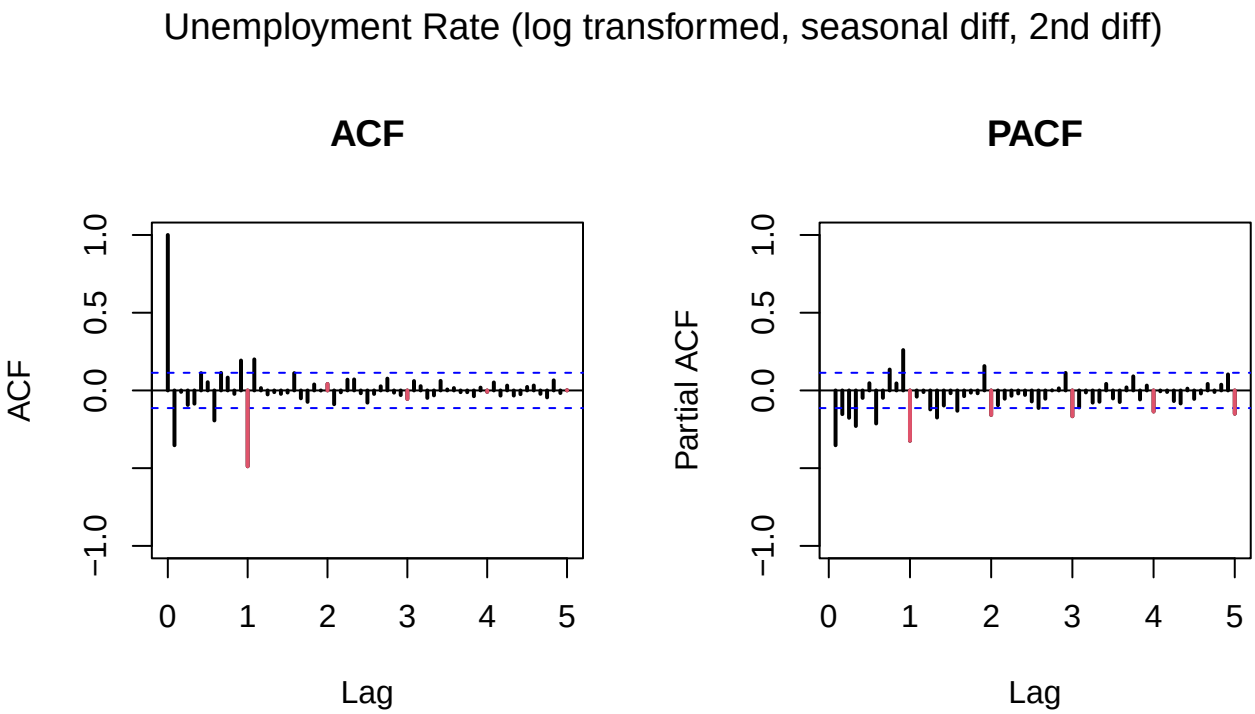
The ACF plot shows slow decay with all lags significant, which suggests non-stationarity and the need for differencing. We apply seasonal and first differencing and re-examine the ACF and PACF plots.

```r
d12ln_transformed <- diff(ln_transformed, lag = 12)
d1d12ln_transformed <- diff(d12ln_transformed)
d1d1d12ln_transformed <- diff(d1d12ln_transformed)
d1d1d1d12ln_transformed <- diff(d1d1d12ln_transformed)

var(d1d12ln_transformed); var(d1d1d12ln_transformed); var(d1d1d1d12ln_transformed)
```

```
##             V1
## V1 0.000349434
##             V1
## V1 0.000326053
##              V1
## V1 0.0008836331
```

Given these results, we need to apply differencing twice, as variance increases on the third difference.

```r
acf_pacf_plots(d1d1d12ln_transformed, paste(series_name, "(log transformed, seasonal diff, 2nd diff)"))
```



The ACF plot shows significant lags at 1 and 12 (seasonal lag 1). The PACF plot exhibits significant lags up to lag 4 with decaying seasonal lags. Based on this pattern, we specify models with: - Non-seasonal AR (p): 0 to 4 - Non-seasonal MA (q): 0 to 1 - Seasonal AR (P): 0 to 1 (because the seasonal lag does not show a clear cutoff, we keep it simple with P=0 or 1) - Seasonal MA (Q): 0 to 1

3 SARIMA Models

3.1 Functions

```
analyze_sarima <- function(series, order, seasonal, verbose = TRUE) {

  s <- frequency(series)
  n <- length(series)
  model_name <- paste0("ARIMA(", paste(order, collapse=","), ")(",
                       paste(seasonal$order, collapse=","), ")", s)

  if (verbose) {
    cat("\n==================================================================\n")
    cat("  ANALYSIS FOR MODEL:", model_name, "\n")
    cat("==================================================================\n")
  }

  # 1. Estimation & Significance Check
  fit_full <- arima(series, order = order, seasonal = seasonal)

  diag_var <- diag(fit_full$var.coef)
  if (any(diag_var <= 0)) {
    if (verbose) {
      cat("\n  CRITICAL WARNING: The Hessian matrix is unstable (Negative Variances).\n")
    }
```

```r
  se_vec <- ifelse(diag_var > 0, sqrt(diag_var), NA)
} else {
  se_vec <- sqrt(diag_var)
}

t_vec  <- fit_full$coef / se_vec
p_vec  <- 2 * (1 - pnorm(abs(t_vec)))

coeftest <- data.frame(
  Estimate = fit_full$coef,
  Std.Error = se_vec,
  t_stat = t_vec,
  p_value = p_vec
)

coeftest$Significant <- ifelse(
  !is.na(coeftest$p_value) &
    (coeftest$p_value < 0.05 | abs(coeftest$t_stat) > 2),
  "Yes", "No"
)

if (verbose) {
  cat("\n[1] COEFFICIENT SIGNIFICANCE (Alpha = 0.05 or |t| > 2):\n")
  print(round(coeftest[,1:4], 4))
  cat("\nSignificance Flags:\n")
  print(coeftest[,5, drop=FALSE])
}

# 2. Residual Analysis (Diagnostics)
resid <- residuals(fit_full)

# A. Causality / Invertibility Roots (Min Modulus)
ar_poly <- fit_full$model$phi
if (length(ar_poly) > 0 && any(ar_poly != 0)) {
  ar_roots_mod <- Mod(polyroot(c(1, -ar_poly)))
  min_ar_root  <- min(ar_roots_mod)
} else {
  min_ar_root  <- NA
}
ma_poly <- fit_full$model$theta
if (length(ma_poly) > 0 && any(ma_poly != 0)) {
  ma_roots_mod <- Mod(polyroot(c(1, ma_poly)))
  min_ma_root  <- min(ma_roots_mod)
} else {
  min_ma_root  <- NA
}

is_causal     <- ifelse(!is.na(min_ar_root), min_ar_root > 1, TRUE)
is_invertible <- ifelse(!is.na(min_ma_root), min_ma_root > 1, TRUE)

if (verbose) {
  cat("\n[2] DIAGNOSTICS:\n")
  cat("Min AR Root Modulus: ", min_ar_root, "\n")
```

```r
  cat("Min MA Root Modulus: ", min_ma_root, "\n")
}

# B. Plots
if (verbose) {
  par(mfrow = c(2, 2), mar = c(4, 4, 3, 2))
  plot(resid, main = "Residuals", ylab = "Error")
  abline(h = 0, col = "gray")
  abline(h = c(-3 * sd(resid), 3 * sd(resid)), lty = 2, col = "red")
  scatter.smooth(sqrt(abs(resid)), main = "Variance Check",
                 ylab = "sqrt(|Resid|)", lpars = list(col = "red"))
  qqnorm(resid)
  qqline(resid, col = "red")
  hist(resid, breaks = 20, freq = FALSE, main = "Histogram")
  curve(dnorm(x, mean = mean(resid), sd = sd(resid)),
        col = "red", add = TRUE)
}

# C. Tests
sw_test <- shapiro.test(resid)
lb_test <- Box.test(resid, lag = 20, type = "Ljung-Box")

# 3. Forecasting & Cross-Validation
train_end <- c(floor(time(series)[n-12]), cycle(series)[n-12])
train_set <- window(series, end = train_end)
test_set  <- window(series, start = time(series)[n-11])

fit_train <- arima(train_set, order = order, seasonal = seasonal)
fc <- predict(fit_train, n.ahead = 12)

fc_orig <- exp(fc$pred); test_orig <- exp(test_set)
lower_bound <- exp(fc$pred - 1.96 * fc$se); upper_bound <- exp(fc$pred + 1.96 * fc$se)

# 4. Summary Table Generation
summary_row <- data.frame(
  Model_Name      = model_name,
  AIC             = fit_full$aic,
  BIC             = BIC(fit_full),
  Sigma2          = fit_full$sigma2,
  Resid_Norm_PVal = sw_test$p.value,
  Normal       = ifelse(sw_test$p.value > 0.05, "Yes", "No"),
  Resid_Ind_PVal  = lb_test$p.value,
  Independent  = ifelse(lb_test$p.value > 0.05, "Yes", "No"),
  Min_AR_Root     = min_ar_root,
  Causal          = ifelse(is_causal, "Yes", "No"),
  Min_MA_Root     = min_ma_root,
  Invertible      = ifelse(is_invertible, "Yes", "No"),
  RMSE            = sqrt(mean((test_orig - fc_orig)^2)),
  RMSPE           = sqrt(mean(((test_orig - fc_orig) / test_orig)^2)) * 100,
  MAE             = mean(abs(test_orig - fc_orig)),
  MAPE_Pct        = mean(abs((test_orig - fc_orig) / test_orig)) * 100,
  Mean_CI_Width   = mean(upper_bound - lower_bound)
)
```

```r
  return(summary_row)
}
```

```r
compare_sarima_models <- function(series, models_list, verbose = TRUE) {
  if (verbose) {
    cat("\n==================================================================\n")
    cat("  STARTING AUTOMATED MODEL COMPARISON\n")
    cat("==================================================================\n")
  }

  comparison_results <- list()
  for (i in seq_along(models_list)) {
    spec <- models_list[[i]]
    if (verbose) {
      cat(paste0("\n==================================================================\n"))
      cat(paste0("  MODEL ", i, ": ARIMA(", paste(spec$order, collapse=","), ")(",
                 paste(spec$seasonal$order, collapse=","), ")", frequency(series), "\n"))
      cat(paste0("==================================================================\n"))
    }
    result <- tryCatch({
      analyze_sarima(series = series,
                     order = spec$order,
                     seasonal = spec$seasonal,
                     verbose = verbose)
    }, error = function(e) {
      cat(paste0("  Error in Model ", i, ": ", e$message, "\n"))
      return(NULL) # Return NULL so we can skip it
    })
    if (!is.null(result)) {
      comparison_results[[length(comparison_results) + 1]] <- result
    }
  }

  final_table <- do.call(rbind, comparison_results)

  cat("\n--- Comparison Complete ---\n")
  return(final_table)
}
```

3.2 Models

We now list all possible models with: - Non-seasonal AR (p): 0 to 4 - Non-seasonal MA (q): 0 to 1 - Seasonal AR (P): 0 to 1 (because the seasonal lag does not show a clear cutoff, we keep it simple with P=0 or 1) - Seasonal MA (Q): 0 to 1 and keeping the differencing order fixed at (d=2, D=1) based on our previous analysis.

```r
full_grid <- expand.grid(p = 0:4, q = 0:1, P = 0:1, Q = 0:1)

# Create all model names first
model_names <- paste0(
  "ARIMA(", full_grid$p, ",2,", full_grid$q, ")",
  "(", full_grid$P, ",1,", full_grid$Q, ")[12]"
)
```

```r
# Create my_models following the same order with names
my_models <- setNames(
  lapply(seq_len(nrow(full_grid)), function(i) {
    list(
      order = c(full_grid$p[i], 2, full_grid$q[i]),
      seasonal = list(order = c(full_grid$P[i], 1, full_grid$Q[i]), period = 12)
    )
  }),
  model_names
)

print(model_names)
```

```
##  [1] "ARIMA(0,2,0)(0,1,0)[12]" "ARIMA(1,2,0)(0,1,0)[12]"
##  [3] "ARIMA(2,2,0)(0,1,0)[12]" "ARIMA(3,2,0)(0,1,0)[12]"
##  [5] "ARIMA(4,2,0)(0,1,0)[12]" "ARIMA(0,2,1)(0,1,0)[12]"
##  [7] "ARIMA(1,2,1)(0,1,0)[12]" "ARIMA(2,2,1)(0,1,0)[12]"
##  [9] "ARIMA(3,2,1)(0,1,0)[12]" "ARIMA(4,2,1)(0,1,0)[12]"
## [11] "ARIMA(0,2,0)(1,1,0)[12]" "ARIMA(1,2,0)(1,1,0)[12]"
## [13] "ARIMA(2,2,0)(1,1,0)[12]" "ARIMA(3,2,0)(1,1,0)[12]"
## [15] "ARIMA(4,2,0)(1,1,0)[12]" "ARIMA(0,2,1)(1,1,0)[12]"
## [17] "ARIMA(1,2,1)(1,1,0)[12]" "ARIMA(2,2,1)(1,1,0)[12]"
## [19] "ARIMA(3,2,1)(1,1,0)[12]" "ARIMA(4,2,1)(1,1,0)[12]"
## [21] "ARIMA(0,2,0)(0,1,1)[12]" "ARIMA(1,2,0)(0,1,1)[12]"
## [23] "ARIMA(2,2,0)(0,1,1)[12]" "ARIMA(3,2,0)(0,1,1)[12]"
## [25] "ARIMA(4,2,0)(0,1,1)[12]" "ARIMA(0,2,1)(0,1,1)[12]"
## [27] "ARIMA(1,2,1)(0,1,1)[12]" "ARIMA(2,2,1)(0,1,1)[12]"
## [29] "ARIMA(3,2,1)(0,1,1)[12]" "ARIMA(4,2,1)(0,1,1)[12]"
## [31] "ARIMA(0,2,0)(1,1,1)[12]" "ARIMA(1,2,0)(1,1,1)[12]"
## [33] "ARIMA(2,2,0)(1,1,1)[12]" "ARIMA(3,2,0)(1,1,1)[12]"
## [35] "ARIMA(4,2,0)(1,1,1)[12]" "ARIMA(0,2,1)(1,1,1)[12]"
## [37] "ARIMA(1,2,1)(1,1,1)[12]" "ARIMA(2,2,1)(1,1,1)[12]"
## [39] "ARIMA(3,2,1)(1,1,1)[12]" "ARIMA(4,2,1)(1,1,1)[12]"
```

3.3 Run Analysis

```r
# Run Analysis
comparison_table <- compare_sarima_models(
  ln_transformed,
  my_models,
  verbose = FALSE
)
print(comparison_table)
```

```
##   Error in Model 1: non-numeric argument to mathematical function
##
## --- Comparison Complete ---
##                Model_Name       AIC       BIC       Sigma2 Resid_Norm_PVal
## 1  ARIMA(1,2,0)(0,1,0)12 -1577.004 -1569.610 0.0002849657    1.777609e-16
## 2  ARIMA(2,2,0)(0,1,0)12 -1581.909 -1570.818 0.0002783962    6.909732e-17
## 3  ARIMA(3,2,0)(0,1,0)12 -1589.441 -1574.653 0.0002695455    6.239173e-17
## 4  ARIMA(4,2,0)(0,1,0)12 -1603.144 -1584.658 0.0002555301    4.250080e-16
## 5  ARIMA(0,2,1)(0,1,0)12 -1597.098 -1589.704 0.0002661666    2.446108e-17
## 6  ARIMA(1,2,1)(0,1,0)12 -1602.646 -1591.555 0.0002594053    3.499962e-17
```

```
## 7  ARIMA(2,2,1)(0,1,0)12 -1615.408 -1600.619 0.0002438282    1.340389e-17
## 8  ARIMA(3,2,1)(0,1,0)12 -1601.095 -1582.609 0.0002573009    1.069209e-16
## 9  ARIMA(4,2,1)(0,1,0)12 -1601.762 -1579.580 0.0002549907    4.144079e-16
## 10 ARIMA(0,2,0)(1,1,0)12 -1622.272 -1614.878 0.0002420332    2.244206e-14
## 11 ARIMA(1,2,0)(1,1,0)12 -1650.611 -1639.520 0.0002188610    5.107204e-15
## 12 ARIMA(2,2,0)(1,1,0)12 -1654.125 -1639.336 0.0002148566    1.454912e-15
## 13 ARIMA(3,2,0)(1,1,0)12 -1663.441 -1644.955 0.0002067118    2.249081e-15
## 14 ARIMA(4,2,0)(1,1,0)12 -1674.991 -1652.809 0.0001974787    4.512146e-14
## 15 ARIMA(0,2,1)(1,1,0)12 -1667.187 -1656.095 0.0002069894    7.050095e-16
## 16 ARIMA(1,2,1)(1,1,0)12 -1678.388 -1663.599 0.0001976893    4.713879e-15
## 17 ARIMA(2,2,1)(1,1,0)12 -1683.600 -1665.115 0.0001904427    2.095419e-15
## 18 ARIMA(3,2,1)(1,1,0)12 -1677.946 -1655.763 0.0001953326    1.404283e-14
## 19 ARIMA(4,2,1)(1,1,0)12 -1677.316 -1651.437 0.0001944481    3.093799e-14
## 20 ARIMA(0,2,0)(0,1,1)12 -1691.520 -1684.125 0.0001813352    7.875315e-15
## 21 ARIMA(1,2,0)(0,1,1)12 -1716.290 -1705.199 0.0001675049    3.557958e-15
## 22 ARIMA(2,2,0)(0,1,1)12 -1718.605 -1703.817 0.0001653383    4.444762e-15
## 23 ARIMA(3,2,0)(0,1,1)12 -1732.069 -1713.583 0.0001567591    7.844648e-16
## 24 ARIMA(4,2,0)(0,1,1)12 -1742.287 -1720.104 0.0001502582    3.952436e-15
## 25 ARIMA(0,2,1)(0,1,1)12 -1731.307 -1720.215 0.0001600327    1.615392e-15
## 26 ARIMA(1,2,1)(0,1,1)12 -1749.257 -1734.469 0.0001478829    1.555145e-15
## 27 ARIMA(2,2,1)(0,1,1)12 -1748.003 -1729.518 0.0001473534    9.143126e-16
## 28 ARIMA(3,2,1)(0,1,1)12 -1749.255 -1727.073 0.0001460655    1.524872e-15
## 29 ARIMA(4,2,1)(0,1,1)12 -1747.256 -1721.376 0.0001460569    1.491170e-15
## 30 ARIMA(0,2,0)(1,1,1)12 -1689.764 -1678.673 0.0001808271    6.127505e-15
## 31 ARIMA(1,2,0)(1,1,1)12 -1715.262 -1700.474 0.0001665210    1.867720e-15
## 32 ARIMA(2,2,0)(1,1,1)12 -1717.651 -1699.166 0.0001643454    1.937677e-15
## 33 ARIMA(3,2,0)(1,1,1)12 -1730.497 -1708.314 0.0001563654    5.409016e-16
## 34 ARIMA(4,2,0)(1,1,1)12 -1741.083 -1715.204 0.0001495947    2.682084e-15
## 35 ARIMA(0,2,1)(1,1,1)12 -1730.566 -1715.777 0.0001590186    6.833109e-16
## 36 ARIMA(1,2,1)(1,1,1)12 -1747.688 -1729.202 0.0001473628    1.110086e-15
## 37 ARIMA(2,2,1)(1,1,1)12 -1746.876 -1724.693 0.0001463383    4.131537e-16
## 38 ARIMA(3,2,1)(1,1,1)12 -1747.468 -1721.588 0.0001458139    1.246620e-15
## 39 ARIMA(4,2,1)(1,1,1)12 -1745.469 -1715.892 0.0001458082    1.222053e-15
##    Normal Resid_Ind_PVal Independent Min_AR_Root Causal Min_MA_Root Invertible
## 1      No   0.000000e+00          No    2.835105    Yes          NA        Yes
## 2      No   1.110223e-16          No    2.569894    Yes          NA        Yes
## 3      No   8.881784e-16          No    1.726662    Yes          NA        Yes
## 4      No   2.517730e-11          No    1.406308    Yes          NA        Yes
## 5      No   6.750156e-14          No          NA    Yes    1.780764        Yes
## 6      No   3.129719e-13          No    3.348311    Yes    1.282584        Yes
## 7      No   4.216627e-13          No    1.456400    Yes    1.000015        Yes
## 8      No   2.634559e-13          No    1.992540    Yes    1.458906        Yes
## 9      No   9.206524e-12          No    1.410363    Yes    3.951145        Yes
## 10     No   2.069472e-06          No    1.058543    Yes          NA        Yes
## 11     No   6.933887e-04          No    1.063789    Yes          NA        Yes
## 12     No   1.419036e-03          No    1.064397    Yes          NA        Yes
## 13     No   1.432179e-03          No    1.063280    Yes          NA        Yes
## 14     No   6.162725e-02         Yes    1.064384    Yes          NA        Yes
## 15     No   2.159284e-03          No    1.066236    Yes    1.874277        Yes
## 16     No   6.542614e-02         Yes    1.062785    Yes    1.241719        Yes
## 17     No   6.144134e-02         Yes    1.066289    Yes    1.000010        Yes
## 18     No   8.401650e-02         Yes    1.062023    Yes    1.421288        Yes
## 19     No   1.571324e-01         Yes    1.062324    Yes    1.780567        Yes
## 20     No   7.324781e-05          No          NA    Yes    1.008399        Yes
```

```
## 21     No   2.065608e-03          No    3.375871    Yes    1.011454        Yes
## 22     No   2.852410e-03          No    2.886448    Yes    1.011986        Yes
## 23     No   1.622178e-02          No    1.602508    Yes    1.011684        Yes
## 24     No   2.059334e-01         Yes    1.385371    Yes    1.011347        Yes
## 25     No   2.986710e-03          No          NA    Yes    1.013473        Yes
## 26     No   2.754285e-01         Yes    2.268389    Yes    1.010233        Yes
## 27     No   3.303534e-01         Yes    1.708332    Yes    1.010451        Yes
## 28     No   4.155792e-01         Yes    1.775555    Yes    1.010186        Yes
## 29     No   4.130882e-01         Yes    1.795383    Yes    1.010175        Yes
## 30     No   5.838690e-05          No    1.328881    Yes    1.007402        Yes
## 31     No   2.049109e-03          No    1.251911    Yes    1.009249        Yes
## 32     No   3.494421e-03          No    1.247199    Yes    1.009649        Yes
## 33     No   1.379343e-02          No    1.295632    Yes    1.010337        Yes
## 34     No   2.431190e-01         Yes    1.262748    Yes    1.009529        Yes
## 35     No   5.399476e-03          No    1.236990    Yes    1.010800        Yes
## 36     No   2.527840e-01         Yes    1.295801    Yes    1.008813        Yes
## 37     No   3.129138e-01         Yes    1.251949    Yes    1.008517        Yes
## 38     No   3.815502e-01         Yes    1.335316    Yes    1.009279        Yes
## 39     No   3.791460e-01         Yes    1.335392    Yes    1.009275        Yes
##            RMSE     RMSPE          MAE  MAPE_Pct Mean_CI_Width
## 1  2.919419e-04 1.1955595 2.727233e-04 1.1116416   0.015638067
## 2  2.295593e-04 0.9382857 2.139217e-04 0.8710928   0.013780568
## 3  1.015243e-04 0.4069275 8.933431e-05 0.3596839   0.012019316
## 4  8.208900e-05 0.3215267 6.390036e-05 0.2541704   0.010228722
## 5  1.169361e-04 0.4701254 1.048965e-04 0.4233640   0.010357927
## 6  1.033672e-04 0.4127146 8.904481e-05 0.3579663   0.009216066
## 7  1.585732e-04 0.6422020 1.476990e-04 0.5985818   0.007298177
## 8  8.449340e-05 0.3322308 6.609105e-05 0.2633081   0.009410045
## 9  8.202571e-05 0.3207277 6.281688e-05 0.2496903   0.009822429
## 10 7.162840e-04 2.9441496 6.492021e-04 2.6535374   0.019218683
## 11 7.428802e-04 3.0537607 6.735679e-04 2.7533615   0.014232493
## 12 8.236295e-04 3.3864033 7.461084e-04 3.0505091   0.012840405
## 13 6.605692e-04 2.7159527 5.948495e-04 2.4322832   0.011115612
## 14 5.012405e-04 2.0601611 4.481997e-04 1.8323452   0.009526304
## 15 6.781146e-04 2.7869541 6.155165e-04 2.5155840   0.009768727
## 16 6.059510e-04 2.4890824 5.519586e-04 2.2547533   0.008353336
## 17 3.241524e-04 1.3220202 3.078087e-04 1.2511862   0.007027739
## 18 4.656939e-04 1.9121372 4.198776e-04 1.7149093   0.008346549
## 19 4.239278e-04 1.7402597 3.793677e-04 1.5494441   0.008373008
## 20 3.818102e-04 1.5504631 3.493581e-04 1.4156259   0.016047115
## 21 3.085753e-04 1.2488519 2.695012e-04 1.0897650   0.012193201
## 22 2.922927e-04 1.1813300 2.491606e-04 1.0062312   0.011074606
## 23 2.716931e-04 1.0946729 2.217948e-04 0.8929772   0.009401181
## 24 2.906145e-04 1.1705011 2.440848e-04 0.9829981   0.008231244
## 25 3.153117e-04 1.2764891 2.772119e-04 1.1213856   0.008427561
## 26 2.844684e-04 1.1462542 2.364185e-04 0.9521956   0.006597481
## 27 2.427607e-04 0.9748196 1.865252e-04 0.7483314   0.006504428
## 28 3.453913e-04 1.3967066 3.087821e-04 1.2480691   0.006744147
## 29 3.412219e-04 1.3795643 3.040563e-04 1.2287476   0.006734102
## 30 3.234751e-04 1.3100463 2.863565e-04 1.1575626   0.015978033
## 31 2.213822e-04 0.8903691 1.784317e-04 0.7174210   0.012058191
## 32 2.109850e-04 0.8477128 1.680659e-04 0.6752113   0.010941930
## 33 2.269394e-04 0.9112161 1.780475e-04 0.7145535   0.009352364
## 34 2.258347e-04 0.9045163 1.800388e-04 0.7216534   0.008152870
```

```
## 35 2.250281e-04 0.9051628 1.742524e-04 0.6999029   0.008332222
## 36 2.517960e-04 1.0123524 2.045732e-04 0.8221767   0.006560716
## 37 2.054025e-04 0.8253266 1.815521e-04 0.7310716   0.006373030
## 38 3.136873e-04 1.2662562 2.735147e-04 1.1037053   0.006734137
## 39 3.097852e-04 1.2502401 2.689103e-04 1.0848915   0.006722479
```

3.4 Choosing Model

From the comparison table we can see that:

- All models are causal and invertible.
- None of the models have normal residuals, likely due to the structural breaks in the data caused by the 2008 financial crisis and the COVID-19 pandemic.
- Less than half of the models have independent residuals.
- The performance metrics produce generally similar ranking across models.

To shortlist, we first select models with independent residuals and then select the top two models based on BIC, which provides a stronger penalty for model complexity than AIC, and on RMSPE, which is preferred over MAPE_pct due to its standardized scale.

```r
# select top 2 models based on BIC and RMSPE
independent_table <- subset(comparison_table, Independent == "Yes")
top_bic <- independent_table[order(independent_table$BIC),][1:2,]
top_rmspe <- independent_table[order(independent_table$RMSPE, decreasing = TRUE),][1:2,]
cat("\nTop 2 Models by BIC:\n")
print(top_bic[, c("Model_Name", "BIC", "RMSPE")])
cat("\nTop 2 Models by RMSPE:\n")
print(top_rmspe[, c("Model_Name", "BIC", "RMSPE")])
```

```
##
## Top 2 Models by BIC:
##               Model_Name       BIC     RMSPE
## 26 ARIMA(1,2,1)(0,1,1)12 -1734.469 1.1462542
## 27 ARIMA(2,2,1)(0,1,1)12 -1729.518 0.9748196
##
## Top 2 Models by RMSPE:
##               Model_Name       BIC    RMSPE
## 16 ARIMA(1,2,1)(1,1,0)12 -1663.599 2.489082
## 14 ARIMA(4,2,0)(1,1,0)12 -1652.809 2.060161
```

4 Model Shortlist

Now we analyze the shortlisted models in more detail.

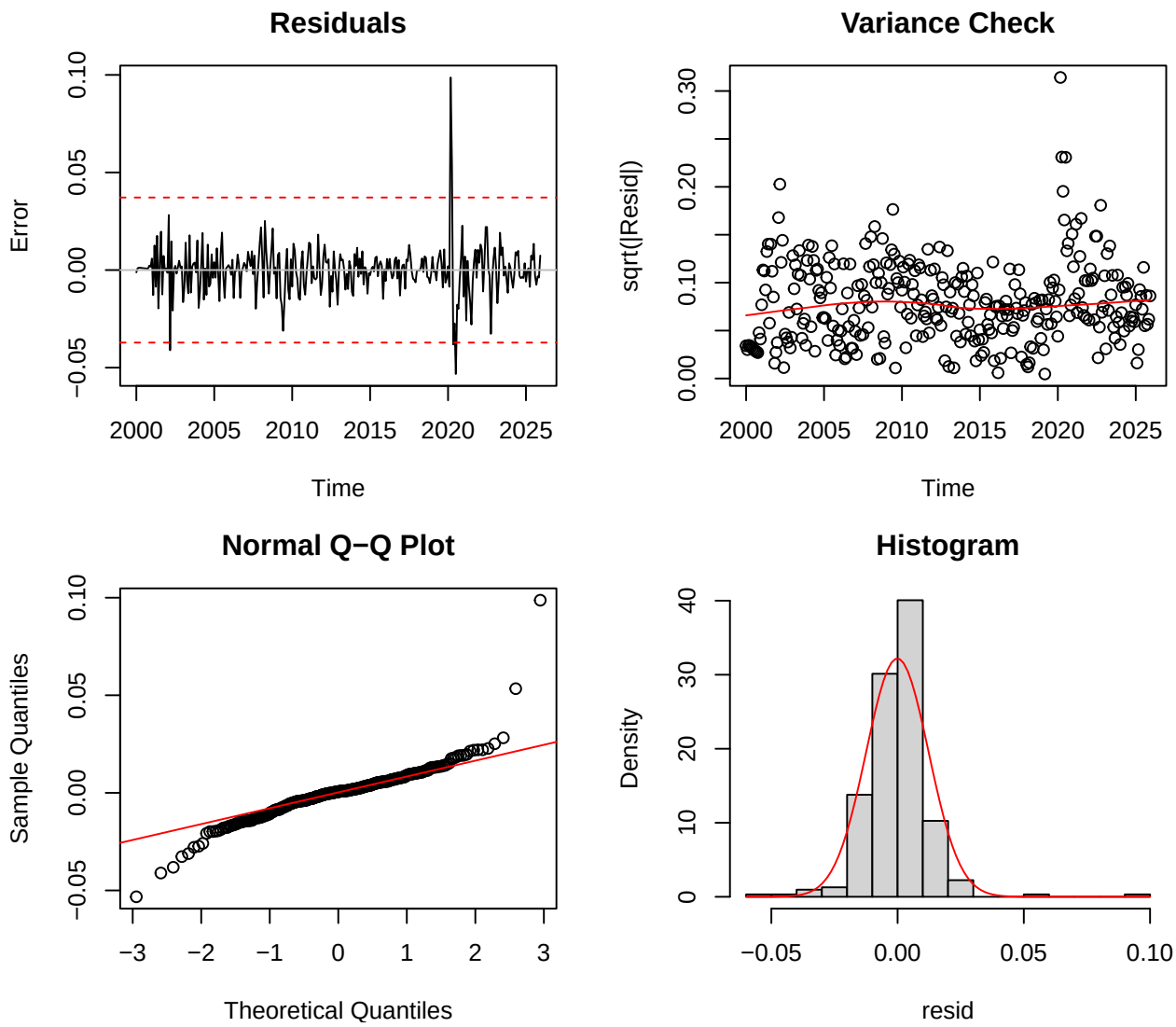
4.1 Function

```r
analyze_sarima_wrapper <- function(model) {
  chosen_model_index <- which(comparison_table$Model_Name == model)
  # get specs from my_models based on index
  chosen_model_spec <- my_models[[chosen_model_index]]
  stats <- analyze_sarima(
    ln_transformed,
    order = chosen_model_spec$order,
    seasonal = chosen_model_spec$seasonal,
    verbose = TRUE
```

```r
)
  return(stats)
}
```

```r
stats <- analyze_sarima_wrapper(top_bic$Model_Name[1])
print(t(stats))
```

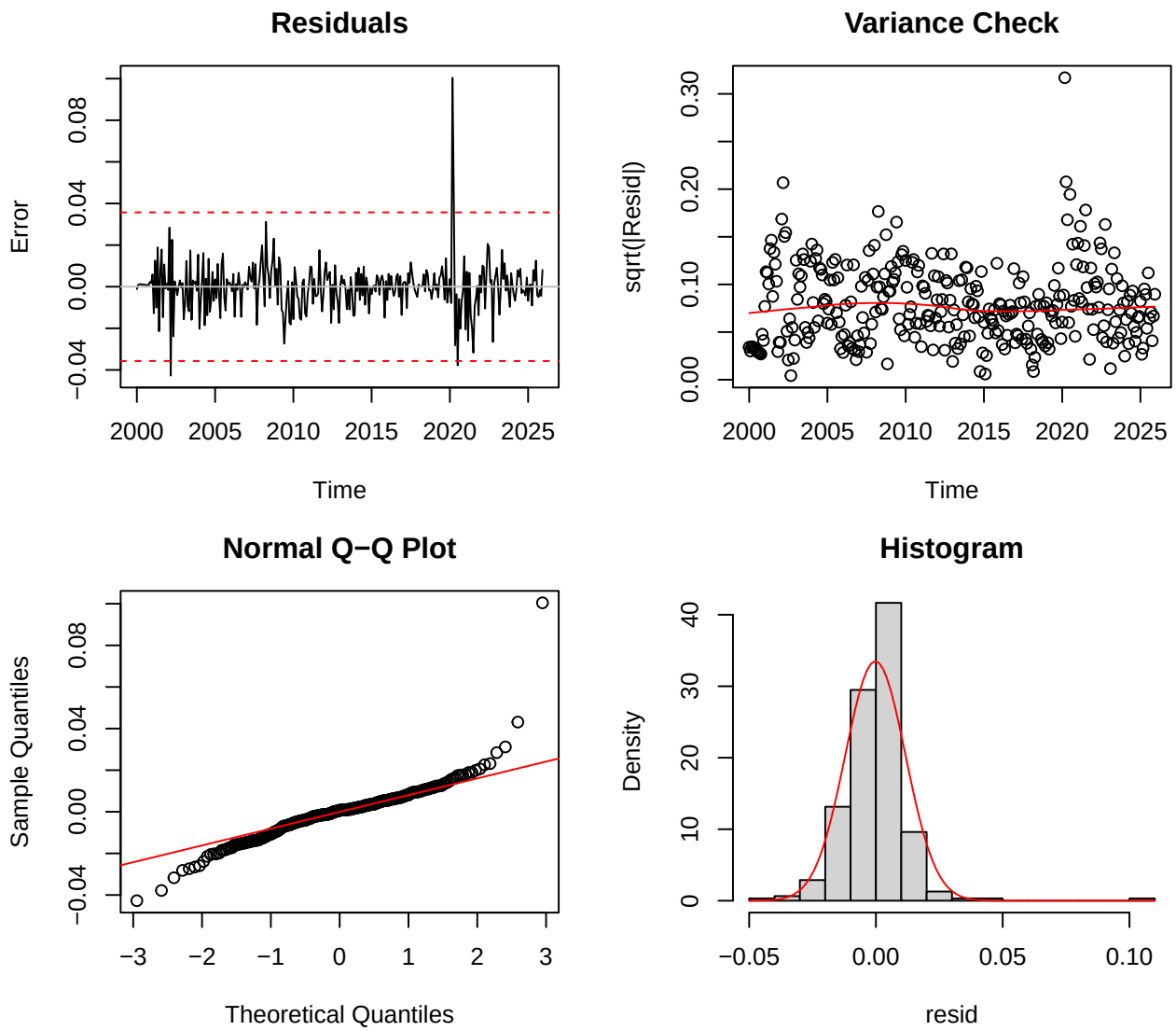
```
##
## ====================================================================
##   ANALYSIS FOR MODEL: ARIMA(0,2,1)(0,1,1)12
## ====================================================================
##
## [1] COEFFICIENT SIGNIFICANCE (Alpha = 0.05 or |t| > 2):
##      Estimate Std.Error   t_stat p_value
## ma1   -0.5274    0.0787  -6.7036       0
## sma1  -0.8516    0.0414 -20.5852       0
##
## Significance Flags:
##      Significant
## ma1          Yes
## sma1         Yes
##
## [2] DIAGNOSTICS:
## Min AR Root Modulus:  NA
## Min MA Root Modulus:  1.013473
##                 [,1]
## Model_Name      "ARIMA(0,2,1)(0,1,1)12"
## AIC             "-1731.307"
## BIC             "-1720.215"
## Sigma2          "0.0001600327"
## Resid_Norm_PVal "1.615392e-15"
## Normal          "No"
## Resid_Ind_PVal  "0.00298671"
## Independent     "No"
## Min_AR_Root     NA
## Causal          "Yes"
## Min_MA_Root     "1.013473"
## Invertible      "Yes"
## RMSE            "0.0003153117"
## RMSPE           "1.276489"
## MAE             "0.0002772119"
## MAPE_Pct        "1.121386"
## Mean_CI_Width   "0.008427561"
```




```r
stats <- analyze_sarima_wrapper(top_bic$Model_Name[2])
print(t(stats))
```

```
##
## ================================================================
##   ANALYSIS FOR MODEL: ARIMA(1,2,1)(0,1,1)12
## ================================================================
##
## [1] COEFFICIENT SIGNIFICANCE (Alpha = 0.05 or |t| > 2):
##      Estimate Std.Error   t_stat p_value
## ar1    0.4408    0.0924   4.7694       0
## ma1   -0.8758    0.0597 -14.6749       0
## sma1  -0.8850    0.0410 -21.5626       0
##
## Significance Flags:
##      Significant
## ar1          Yes
## ma1          Yes
## sma1         Yes
```

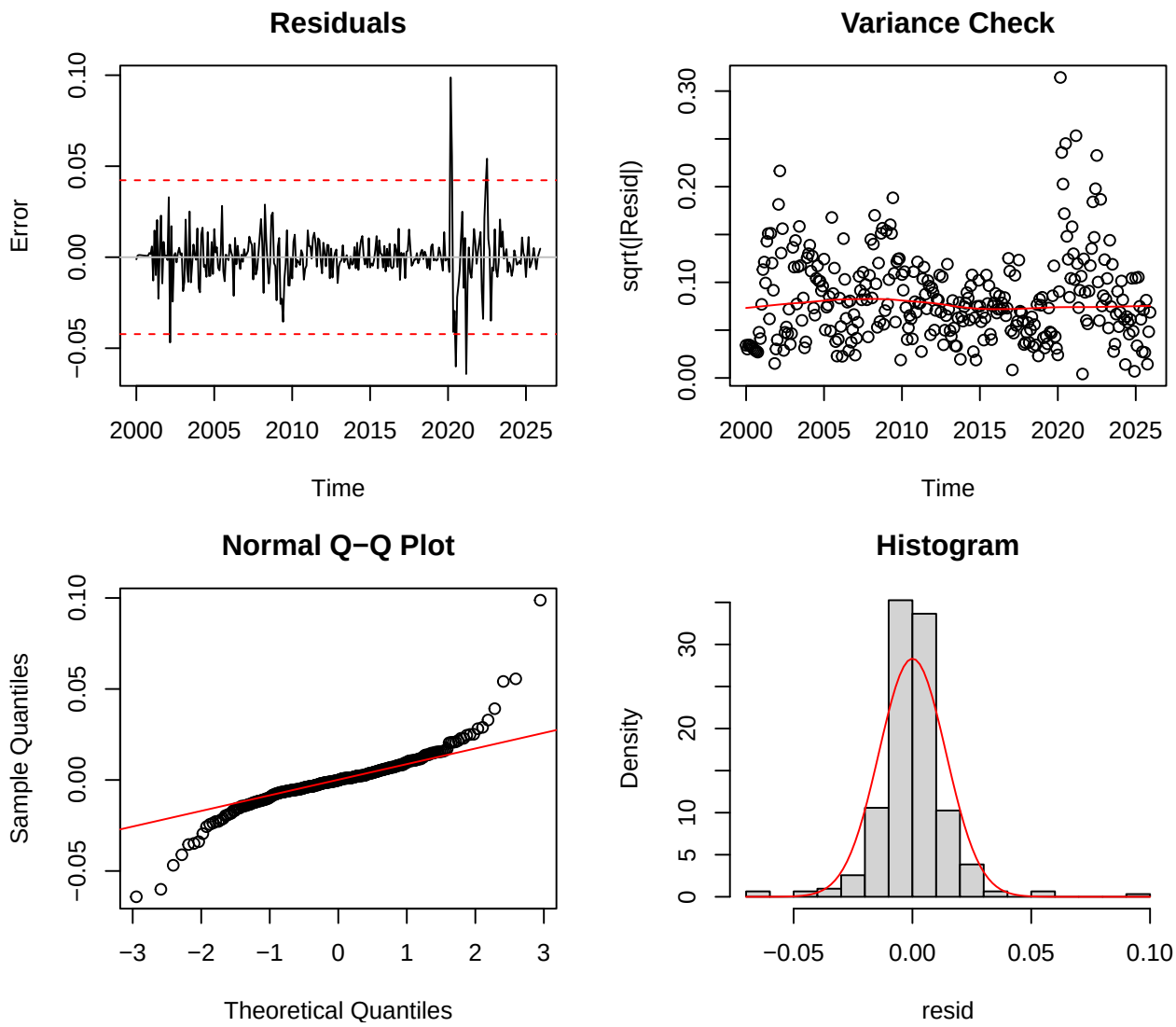
```
##
## [2] DIAGNOSTICS:
## Min AR Root Modulus:  2.268389
## Min MA Root Modulus:  1.010233
##                  [,1]
## Model_Name       "ARIMA(1,2,1)(0,1,1)12"
## AIC              "-1749.257"
## BIC              "-1734.469"
## Sigma2           "0.0001478829"
## Resid_Norm_PVal "1.555145e-15"
## Normal           "No"
## Resid_Ind_PVal  "0.2754285"
## Independent      "Yes"
## Min_AR_Root      "2.268389"
## Causal           "Yes"
## Min_MA_Root      "1.010233"
## Invertible       "Yes"
## RMSE             "0.0002844684"
## RMSPE            "1.146254"
## MAE              "0.0002364185"
## MAPE_Pct         "0.9521956"
## Mean_CI_Width    "0.006597481"
```




```r
stats <- analyze_sarima_wrapper(top_rmspe$Model_Name[1])
print(t(stats))
```

```
##
## ================================================================
##   ANALYSIS FOR MODEL: ARIMA(0,2,1)(1,1,0)12
## ================================================================
##
## [1] COEFFICIENT SIGNIFICANCE (Alpha = 0.05 or |t| > 2):
##      Estimate Std.Error  t_stat p_value
## ma1   -0.5335    0.0704 -7.5806       0
## sar1  -0.4632    0.0507 -9.1350       0
##
## Significance Flags:
##      Significant
## ma1          Yes
## sar1         Yes
##
## [2] DIAGNOSTICS:
```

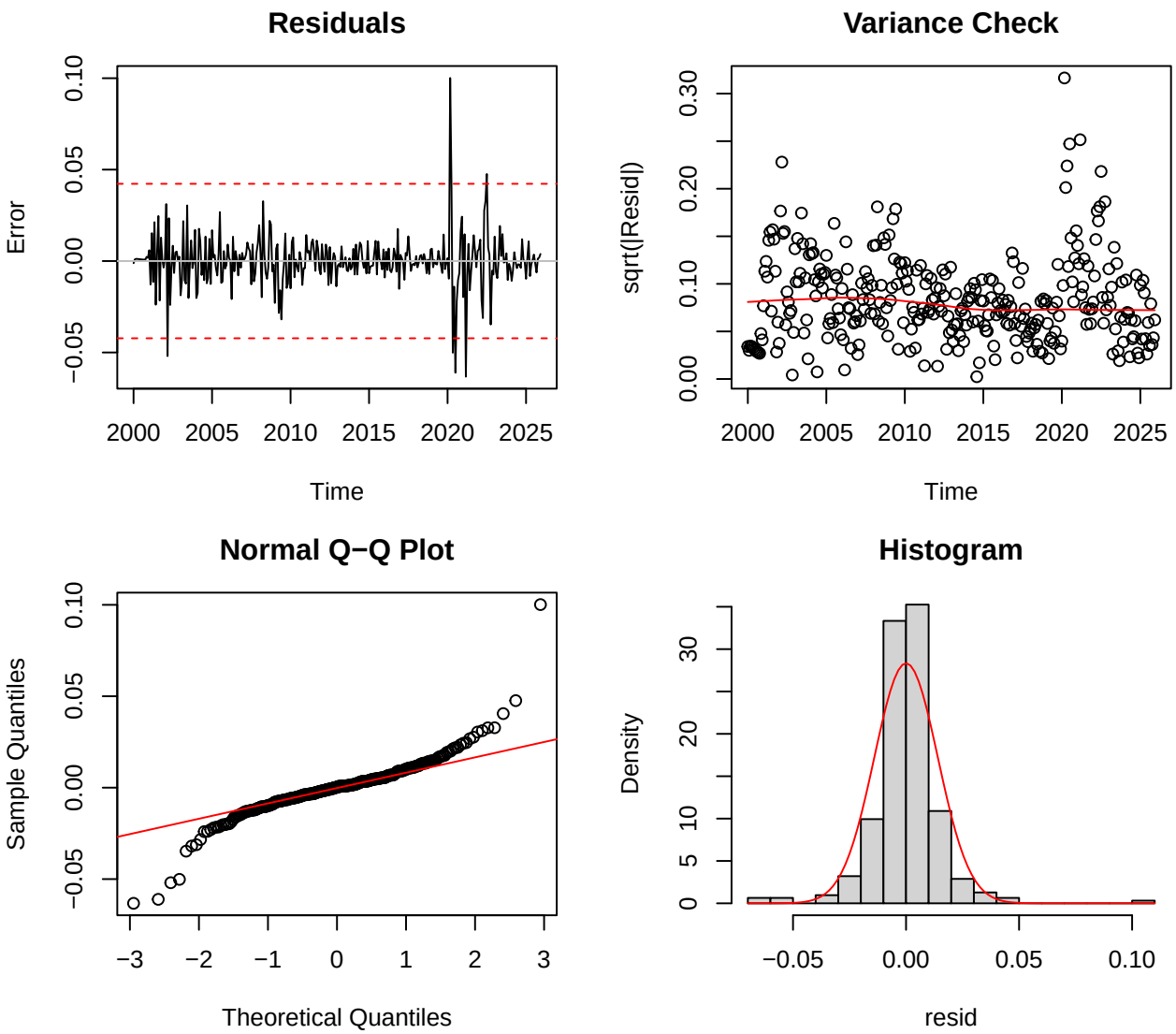
```
## Min AR Root Modulus:  1.066236
## Min MA Root Modulus:  1.874277
##                  [,1]
## Model_Name       "ARIMA(0,2,1)(1,1,0)12"
## AIC              "-1667.187"
## BIC              "-1656.095"
## Sigma2           "0.0002069894"
## Resid_Norm_PVal "7.050095e-16"
## Normal           "No"
## Resid_Ind_PVal   "0.002159284"
## Independent      "No"
## Min_AR_Root      "1.066236"
## Causal           "Yes"
## Min_MA_Root      "1.874277"
## Invertible       "Yes"
## RMSE             "0.0006781146"
## RMSPE            "2.786954"
## MAE              "0.0006155165"
## MAPE_Pct         "2.515584"
## Mean_CI_Width    "0.009768727"
```




```r
stats <- analyze_sarima_wrapper(top_rmspe$Model_Name[2])
print(t(stats))
```

```
##
## ================================================================
##   ANALYSIS FOR MODEL: ARIMA(3,2,0)(1,1,0)12
## ================================================================
##
## [1] COEFFICIENT SIGNIFICANCE (Alpha = 0.05 or |t| > 2):
##      Estimate Std.Error  t_stat p_value
## ar1   -0.3814    0.0573 -6.6545   0e+00
## ar2   -0.2041    0.0599 -3.4083   7e-04
## ar3   -0.1928    0.0568 -3.3973   7e-04
## sar1  -0.4789    0.0508 -9.4252   0e+00
##
## Significance Flags:
##      Significant
## ar1          Yes
## ar2          Yes
```

```
## ar3          Yes
## sar1         Yes
##
## [2] DIAGNOSTICS:
## Min AR Root Modulus:  1.06328
## Min MA Root Modulus:  NA
##                 [,1]
## Model_Name      "ARIMA(3,2,0)(1,1,0)12"
## AIC             "-1663.441"
## BIC             "-1644.955"
## Sigma2          "0.0002067118"
## Resid_Norm_PVal "2.249081e-15"
## Normal          "No"
## Resid_Ind_PVal  "0.001432179"
## Independent     "No"
## Min_AR_Root     "1.06328"
## Causal          "Yes"
## Min_MA_Root     NA
## Invertible      "Yes"
## RMSE            "0.0006605692"
## RMSPE           "2.715953"
## MAE             "0.0005948495"
## MAPE_Pct        "2.432283"
## Mean_CI_Width   "0.01111561"
```



4.2 Final Model: ARIMA(1,2,1)(0,1,1)[12]

In choosing the final model, we examine the following criteria:

Residual Analysis: Across all models, the residuals fail the normality test, evident in the Q–Q plots. However, the residual plots and histograms show that this deviation is largely driven by extreme observations during the COVID-19 period. The 2008 financial crisis also shows elevated residuals, though less extreme. One additional extreme residual appears between 2000 and 2005 that cannot be linked to any identifiable event. The variance decreases slightly over time but the slope is minimal, suggesting reasonable stability and not a deciding factor.

Coefficient Significance: All coefficients across shortlisted models are significant so this is not a deciding factor.

Model Parsimony: The most parsimonious models are:

- ARIMA(1,2,1)(1,1,0)
- ARIMA(1,2,1)(0,1,1)[12]

Model Performance: ARIMA(1,2,1)(0,1,1)[12] demonstrates the best performance.

Therefore, we select **ARIMA(1,2,1)(0,1,1)[12]** as our final model for unemployment forecasting.